

Multiple Frequencies, High Dimensional Models, Ridge and Lasso.

STAT 429: Introduction to Time Series
Week 5

Sabyasachi Chatterjee

Contents

1	Sinusoidal Models with more frequencies	2
1.0.1	Restriction to Fourier Frequencies	2
1.0.2	Orthogonality of Sinusoidal Vectors at Fourier Frequencies	3
1.0.3	Proof of (2)	5
1.0.4	More than two Fourier frequencies	5
1.1	Other Nonlinear Models	6
2	High Dimensional Models	6
2.1	High-dimensional Version of (5)	7
2.1.1	Least Squares in (7) interpolates data	8
2.2	High-dimensional Linear Model	9
2.3	Two alternative representations of (10)	9
2.4	(Unregularized) Least Squares	9
3	Ridge and Lasso Regularization	10
3.1	Alternative Representations	10
3.2	Ridge vs LASSO	11
3.3	Cross-validation for selecting λ	12

STAT 429: Introduction to Time Series Analysis

Week 5 Lecture Notes

STAT 429: Introduction to Time Series
Week 5

Sabyasachi Chatterjee

1 Sinusoidal Models with more frequencies

Consider the model:

$$y_t = \beta_0 + \beta_1 \cos(2\pi f_1 t) + \beta_2 \sin(2\pi f_1 t) + \beta_3 \cos(2\pi f_2 t) + \beta_4 \sin(2\pi f_2 t) + \epsilon_t. \quad (1)$$

Formal inference for this model proceeds very similarly to inference for model (??). The main difference is that the definition of RSS should now be changed to:

$$RSS(f_1, f_2) = \min_{\beta_j, 0 \leq j \leq 4} \sum_{t=0}^{n-1} (y_t - \beta_0 - \beta_1 \cos(2\pi f_1 t) - \beta_2 \sin(2\pi f_1 t) - \beta_3 \cos(2\pi f_2 t) - \beta_4 \sin(2\pi f_2 t))^2$$

The analysis now proceeds as before with this modified definition of RSS . The least squares estimates of f_1 and f_2 are obtained by minimizing $RSS(f_1, f_2)$ over f_1, f_2 , and the Bayesian posterior is given by

$$\propto \left(\frac{1}{RSS(f_1, f_2)} \right)^{(n-5)/2} |X_{f_1, f_2}^T X_{f_1, f_2}|^{-1/2}$$

where X_{f_1, f_2} is now given by

$$X_{f_1, f_2} = \begin{pmatrix} 1 & \cos(2\pi f_1(0)) & \sin(2\pi f_1(0)) & \cos(2\pi f_2(0)) & \sin(2\pi f_2(0)) \\ 1 & \cos(2\pi f_1(1)) & \sin(2\pi f_1(1)) & \cos(2\pi f_2(1)) & \sin(2\pi f_2(1)) \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & \cos(2\pi f_1(n-1)) & \sin(2\pi f_1(n-1)) & \cos(2\pi f_2(n-1)) & \sin(2\pi f_2(n-1)) \end{pmatrix}$$

Evaluation and minimization of $RSS(f_1, f_2)$ needs to be done on a joint grid for f_1 and f_2 .

1.0.1 Restriction to Fourier Frequencies

Suppose we restrict f_1 and f_2 to be distinct Fourier frequencies lying strictly between 0 and 0.5. Then it can be proved that

$$RSS(f_1, f_2) = \sum_{t=0}^{n-1} (y_t - \bar{y})^2 - 2I(f_1) - 2I(f_2). \quad (2)$$

From the above, it is clear that f_1 and f_2 which minimize $RSS(f_1, f_2)$ equal the top two maximizers of the periodogram (remember that we are assuming f_1 and f_2 to be distinct).

The equality (2) is a consequence of the orthogonality of sinusoidal vectors corresponding to Fourier frequencies. It is proved below.

1.0.2 Orthogonality of Sinusoidal Vectors at Fourier Frequencies

Orthogonality of Sinusoids at Fourier frequencies is best described through complex sinusoids.

For every $0 \leq j \leq (n-1)$, let us define the $n \times 1$ vector

$$u^j = (1, \exp(2\pi i j/n), \exp(2\pi i 2j/n), \dots, \exp(2\pi i (n-1)j/n))^T.$$

This vector can be interpreted as the complex sinusoid $e^{2\pi i f t}$ with Fourier frequency $f = j/n$ evaluated at the time points $t = 0, 1, \dots, (n-1)$. It is easy to see that

1. When $j = 0$, we have $u^0 = (1, 1, \dots, 1)$.
2. When $1 \leq j \leq n-1$, we have $u^j = \overline{u^{n-j}}$. Here $\bar{u}$ denotes complex conjugate of u (the complex conjugate $\bar{u}$ of a vector u is defined as the vector obtained by taking the complex conjugates of each entry of u).

The most important property of these complex valued vectors $u^0, u^1, \dots, u^{n-1}$ is **orthogonality**. Specifically, for $0 \leq j \neq k \leq n-1$, we have

$$\langle u^j, u^k \rangle = 0. \quad (3)$$

Recall that the inner product between two complex valued vectors $a = (a_1, \dots, a_n)^T$ and $b = (b_1, \dots, b_n)^T$ is given by

$$\langle a, b \rangle = \sum_{j=1}^n a_j \bar{b}_j.$$

Note specially the complex conjugate of b_j above.

Here is the proof of (3). Fix $0 \leq j \neq k \leq n-1$ and write

$$\begin{aligned} \langle u^j, u^k \rangle &= \sum_{t=0}^{n-1} \exp\left(2\pi i \frac{j}{n} t\right) \overline{\exp\left(2\pi i \frac{k}{n} t\right)} \\ &= \sum_{t=0}^{n-1} \exp\left(2\pi i \frac{j}{n} t\right) \exp\left(-2\pi i \frac{k}{n} t\right) \\ &= \sum_{t=0}^{n-1} \exp\left(2\pi i \frac{j-k}{n} t\right) \\ &= \sum_{t=0}^{n-1} \left[\exp\left(2\pi i \frac{j-k}{n}\right) \right]^t \\ &= \frac{1 - \left(\exp\left(2\pi i \frac{j-k}{n}\right)\right)^n}{1 - \exp\left(2\pi i \frac{j-k}{n}\right)} \\ &= \frac{1 - \exp(2\pi i(j-k))}{1 - \exp\left(2\pi i \frac{j-k}{n}\right)} \\ &= \frac{1 - \cos(2\pi(j-k)) - i \sin(2\pi(j-k))}{1 - \exp\left(2\pi i \frac{j-k}{n}\right)} = \frac{1 - 1 - 0}{1 - \exp\left(2\pi i \frac{j-k}{n}\right)} = 0 \end{aligned}$$

This proves (3). It is also easy to see that (just take $j = k$ in the above calculation and the answer can be found in the third line)

$$\langle u^j, u^j \rangle = \|u^j\|^2 = n.$$

This orthogonality of complex sinusoids also extends to real sinusoids made of sines and cosines. For a Fourier frequency j/n strictly lying between 0 and 1 (i.e., $0 < j/n < 1/2$), define

$$\mathbf{c}^j := (1, \cos(2\pi j/n), \cos(2\pi 2j/n), \dots, \cos(2\pi(n-1)j/n))^T$$

and

$$\mathbf{s}^j := (0, \sin(2\pi j/n), \sin(2\pi 2j/n), \dots, \sin(2\pi(n-1)j/n))^T.$$

These are the vectors obtained by evaluating $\cos(2\pi ft)$ and $\sin(2\pi ft)$ with Fourier Frequency $f = j/n$ at time points $t = 0, 1, \dots, (n-1)$. So far we have assumed that $0 < f = j/n < 1/2$ i.e., $0 < j < n/2$. The definition can be extended to the boundary points 0 and $1/2$ as well.

When $j = 0$, the vector $\mathbf{c}^0$ is the vector of all ones, while $\mathbf{s}^0$ equals the zero vector. When $f = 1/2$ (this is only when n is even other $1/2$ will not be a Fourier frequency) or $j = n/2$, we have

$$\mathbf{c}^{n/2} = (1, -1, 1, -1, \dots, (-1)^{n-1}) \quad \text{and} \quad \mathbf{s}^{n/2} = (0, \dots, 0).$$

Thus when n is even, the non-zero vectors among these are:

$$\mathbf{c}^0, \mathbf{c}^1, \mathbf{s}^1, \dots, \mathbf{c}^{\frac{n}{2}-1}, \mathbf{s}^{\frac{n}{2}-1}, \mathbf{c}^{\frac{n}{2}}.$$

When n is odd, we are looking at

$$\mathbf{c}^0, \mathbf{c}^1, \mathbf{s}^1, \dots, \mathbf{c}^{\frac{n-1}{2}}, \mathbf{s}^{\frac{n-1}{2}}.$$

In either case, the total number of these vectors equals n .

These vectors are orthogonal in $\mathbb{C}^n$. This can be proved as a consequence of the orthogonality of $u^0, \dots, u^{n-1}$, and the facts:

$$\mathbf{c}^j = \frac{u^j + \overline{u^j}}{2} = \frac{u^j + u^{n-j}}{2} \quad \text{and} \quad \mathbf{s}^j = \frac{u^j - \overline{u^j}}{2i} = \frac{u^j - u^{n-j}}{2i}$$

For example, fix two distinct Fourier frequencies j/n and k/n which are both strictly in $(0, 1/2)$. Then

$$\begin{aligned} \langle \mathbf{c}^j, \mathbf{c}^k \rangle &= \left\langle \frac{u^j + u^{n-j}}{2}, \frac{u^k + u^{n-k}}{2} \right\rangle \\ &= \frac{1}{4} \left(\langle u^j, u^k \rangle + \langle u^j, u^{n-k} \rangle + \langle u^{n-j}, u^k \rangle + \langle u^{n-j}, u^{n-k} \rangle \right). \end{aligned}$$

Each inner product above equals zero because of orthogonality of $u^0, \dots, u^{n-1}$. Indeed, $\langle u^j, u^k \rangle$ and $\langle u^{n-j}, u^{n-k} \rangle$ are zero because $j \neq k$. Also we assumed that $j/n < 0.5$ and $k/n < 0.5$ so that $j+k < n$ which means that $n-j \neq k$ and also $n-k \neq j$. Thus $\langle u^j, u^{n-k} \rangle$ and $\langle u^{n-j}, u^k \rangle$ are also zero. Therefore: $\langle \mathbf{c}^j, \mathbf{c}^k \rangle = 0$ for $j \neq k$ (it can be easily checked that this will be true even when j/n or k/n equal 0 or $1/2$).

One can similarly prove that other inner products (such as those between sines) also equal zero. As another example

$$\langle \mathbf{c}^j, \mathbf{s}^j \rangle = \left\langle \frac{u^j + u^{n-j}}{2}, \frac{u^j - u^{n-j}}{2i} \right\rangle = \frac{1}{4i} [\langle u^j, u^j \rangle - \langle u^{n-j}, u^{n-j} \rangle] = \frac{1}{4i} (n - n) = 0$$

where we used $\langle u^j, u^j \rangle = n$ for every j .

1.0.3 Proof of (2)

Suppose $f_1 = j/n$ and $f_2 = k/n$ with $j \neq k$ and both $0 < j/n, k/n < 0.5$. Then note that X_{f_1, f_2} is a $n \times 5$ matrix with columns $\mathbf{c}^0, \mathbf{c}^j, \mathbf{s}^j, \mathbf{c}^k, \mathbf{s}^k$. As a result,

$$\begin{aligned} X_{f_1, f_2}^T X_{f_1, f_2} &= \begin{pmatrix} \langle \mathbf{c}^0, \mathbf{c}^0 \rangle & \langle \mathbf{c}^0, \mathbf{c}^j \rangle & \langle \mathbf{c}^0, \mathbf{s}^j \rangle & \langle \mathbf{c}^0, \mathbf{c}^k \rangle & \langle \mathbf{c}^0, \mathbf{s}^k \rangle \\ \langle \mathbf{c}^j, \mathbf{c}^0 \rangle & \langle \mathbf{c}^j, \mathbf{c}^j \rangle & \langle \mathbf{c}^j, \mathbf{s}^j \rangle & \langle \mathbf{c}^j, \mathbf{c}^k \rangle & \langle \mathbf{c}^j, \mathbf{s}^k \rangle \\ \langle \mathbf{s}^j, \mathbf{c}^0 \rangle & \langle \mathbf{s}^j, \mathbf{c}^j \rangle & \langle \mathbf{s}^j, \mathbf{s}^j \rangle & \langle \mathbf{s}^j, \mathbf{c}^k \rangle & \langle \mathbf{s}^j, \mathbf{s}^k \rangle \\ \langle \mathbf{c}^k, \mathbf{c}^0 \rangle & \langle \mathbf{c}^k, \mathbf{c}^j \rangle & \langle \mathbf{c}^k, \mathbf{s}^j \rangle & \langle \mathbf{c}^k, \mathbf{c}^k \rangle & \langle \mathbf{c}^k, \mathbf{s}^k \rangle \end{pmatrix} \\ &= \begin{pmatrix} \langle \mathbf{c}^0, \mathbf{c}^0 \rangle & 0 & 0 & 0 & 0 \\ 0 & \langle \mathbf{c}^j, \mathbf{c}^j \rangle & 0 & 0 & 0 \\ 0 & 0 & \langle \mathbf{s}^j, \mathbf{s}^j \rangle & 0 & 0 \\ 0 & 0 & 0 & \langle \mathbf{c}^k, \mathbf{c}^k \rangle & 0 \\ 0 & 0 & 0 & 0 & \langle \mathbf{s}^k, \mathbf{s}^k \rangle \end{pmatrix} \end{aligned}$$

It is also easy to see that

$$\langle \mathbf{c}^0, \mathbf{c}^0 \rangle = n \text{ and } \langle \mathbf{c}^j, \mathbf{c}^j \rangle = \langle \mathbf{s}^j, \mathbf{s}^j \rangle = \langle \mathbf{c}^k, \mathbf{c}^k \rangle = \langle \mathbf{s}^k, \mathbf{s}^k \rangle = n/2.$$

They also follow from properties of the complex sinusoidal vectors u^j . For example,

$$\begin{aligned} \langle \mathbf{c}^j, \mathbf{c}^j \rangle &= \left\langle \frac{u^j + u^{n-j}}{2}, \frac{u^j + u^{n-j}}{2} \right\rangle \\ &= \frac{1}{4} (\langle u^j, u^j \rangle + \langle u^j, u^{n-j} \rangle + \langle u^{n-j}, u^j \rangle + \langle u^{n-j}, u^{n-j} \rangle) \\ &= \frac{1}{4} (n + 0 + 0 + n) = \frac{n}{2} \end{aligned}$$

Thus

$$X_{f_1, f_2}^T X_{f_1, f_2} = \begin{pmatrix} n & 0 & 0 & 0 & 0 \\ 0 & n/2 & 0 & 0 & 0 \\ 0 & 0 & n/2 & 0 & 0 \\ 0 & 0 & 0 & n/2 & 0 \\ 0 & 0 & 0 & 0 & n/2 \end{pmatrix}$$

From here the proof proceeds in the same way as in the case of a single Fourier frequency to yield (2).

1.0.4 More than two Fourier frequencies

When there are three or more Fourier frequencies, the formula (2) generalizes in the same way. For example, if f_1, f_2, f_3 are all distinct Fourier frequencies strictly lying between 0 and $1/2$, then

$$RSS(f_1, f_2, f_3) = \sum_{t=0}^{n-1} (y_t - \bar{y})^2 - 2I(f_1) - 2I(f_2) - 2I(f_3).$$

So if we are trying to find the three best Fourier frequencies which best fit the data, we simply pick the top three maximizers of the periodogram. If we do not restrict to Fourier frequencies however, we have to do a harder (say grid based) minimization of $RSS(f_1, f_2, f_3)$. Depending on the application, there could be significant gains in RSS if we go beyond Fourier frequencies.

1.1 Other Nonlinear Models

Our methodology for inference in these sinusoid models also extends to other similar nonlinear regression models. For example, consider the following model which is applicable when we want to introduce two break points for the regression line:

$$y_t = \beta_0 + \beta_1 t + \beta_2(t - c_1) + \beta_3(t - c_2) + \epsilon_t$$

with $\epsilon_t \stackrel{\text{i.i.d.}}{\sim} N(0, \sigma^2)$. This model can also be written as

$$y = X_c \beta + \epsilon$$

where $c = (c_1, c_2)$ and

$$X_c := \begin{pmatrix} 1 & 1 & (1 - c_1) & (1 - c_2) \\ 1 & 2 & (2 - c_1) & (2 - c_2) \\ 1 & 3 & (3 - c_1) & (3 - c_2) \\ \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots \\ 1 & n & (n - c_1) & (n - c_2) \end{pmatrix}$$

We estimate $c = (c_1, c_2)$ by minimizing $RSS(c)$ over all $c = (c_1, c_2)$ with $c_1, c_2 \in [1, n]$, and

$$RSS(c) = \min_{\beta} \|y - X_c \beta\|^2.$$

One can numerically minimize $RSS(c)$ over all $c_1, c_2 \in [1, n]$. A natural grid one can use here is $\{1, \dots, n\}$.

The posterior of c becomes:

$$\pi(c \mid \text{data}) \propto \left(\frac{1}{RSS(c)} \right)^{(n-4)/2} |X_c^T X_c|^{-1/2}.$$

For more break points, one can consider:

$$y_t = \beta_0 + \beta_1 t + \sum_{j=1}^k \beta_{j+1}(t - c_j) + \epsilon_t.$$

Conceptually estimation and inference here proceed just as before with X_c changed appropriately. But the method can become computationally expensive if $k \geq 4$.

2 High Dimensional Models

We will now start our third main topic: High-dimensional linear regression. We shall motivate our basic high-dimensional linear regression model starting from the change of slope (or broken stick regression) model that we previously considered.

The change of slope (or broken stick regression) model is given by:

$$y_t = \beta_0 + \beta_1 t + \beta_2(t - c)_+ + \epsilon_t. \quad (4)$$

Recall that $(t - c)_+$ equals 0 if $t \leq c$ and $t - c$ if $t \geq c$. We shall also sometimes write $(t - c)_+ = (t - c)$. The model (4) says that for times $t \leq c$, the slope of the regression line

is β_1 while for $t > c$, the slope changes to $\beta_1 + \beta_2$. Note β_2 represents the change in slopes (after $t = c$ and before $t = c$).

Generalizations of model (4) can be obtained by having k points of change of slope (instead of just one):

$$y_t = \beta_0 + \beta_1 t + \beta_2(t - c_1) + \beta_3(t - c_2) + \cdots + \beta_{k+1}(t - c_k) + \epsilon_t. \quad (5)$$

We use least squares to estimate the unknown parameters $c_1, \dots, c_k, \beta_0, \beta_1, \dots, \beta_{k+1}$ in this model. We recommended the method where we first compute the function:

$$RSS(c_1, \dots, c_k) := \min_{\beta_0, \dots, \beta_{k+1}} \sum_{t=1}^n (y_t - \beta_0 - \beta_1 t - \beta_2(t - c_1) - \cdots - \beta_{k+1}(t - c_k))^2$$

and then attempt to minimize $RSS(c_1, \dots, c_k)$ by some grid-based search jointly over $c_1, \dots, c_k$ to estimate $c_1, \dots, c_k$. Once $c_1, \dots, c_k$ are estimated by $\hat{c}_1, \dots, \hat{c}_k$, we estimate the remaining parameters $\beta_0, \dots, \beta_{k+1}$ by least squares in the linear regression model obtained by fixing each c_j by $\hat{c}_j$.

This method is computationally inefficient especially when k is not small (even when $k \geq 3$). An alternative method of obtaining the least squares estimates is to directly minimize the least squares objective with respect to all parameters:

$$\min_{\beta_0, \dots, \beta_{k+1}, c_1, \dots, c_k} \sum_{t=1}^n (y_t - \beta_0 - \beta_1 t - \beta_2(t - c_1) - \cdots - \beta_{k+1}(t - c_k))^2. \quad (6)$$

This is a non-trivial optimization problem but several optimization libraries provide functions which can be employed for solving this. One option is the PyTorch library in Python which uses first order optimization algorithms (gradient descent and related methods). These methods require a good initialization for the parameters (for $c_1, \dots, c_k$, a natural initialization is to take equally spaced quantiles of t ; once c_j 's are chosen, initial values for $\beta_0, \dots, \beta_{k+1}$ can be obtained by running a linear regression with those fixed c_j 's).

When k is not small, the least squares estimates (6) can lead to fitted values which overfit the data. In such cases, we need to add a regularization term to the objective function to prevent overfitting. We shall see how to do this in the case of the high-dimensional model which uses all possible c_j 's.

2.1 High-dimensional Version of (5)

We illustrate overfitting prevention via regularization in the high-dimensional model obtained by using all possible c_j 's in (5). This model is described below. Note that because our time t runs from $1, \dots, n$, we can restrict each c_j to the open interval $(1, n)$. Indeed, when $c_j \leq 1$, the ReLU term $(t - c_j)$ coincides with the linear term $t - c_j$ (which can be absorbed in $\beta_0 + \beta_1 t$) and when $c_j \geq n$, the ReLU term $(t - c_j)$ equals zero.

In fact, it turns out that it is enough to restrict c_j to be an integer in $(1, n)$ i.e., $c_j \in \{2, \dots, n-1\}$. This is because $(t - c_j), t = 1, \dots, n$ for any other c_j can be rewritten as a linear combination of $(t - c_j), t = 1, \dots, n$ for integer c_j . As an example, note that

$$(t - 5.7) = 0.3(t - 5) + 0.7(t - 6) \quad \text{for all } t \leq 5 \text{ and } t \geq 6.$$

Allowing all possible c_j 's in $\{2, \dots, n-1\}$ leads to the model:

$$y_t = \beta_0 + \beta_1(t - 1) + \beta_2(t - 2) + \cdots + \beta_{n-1}(t - (n - 1)) + \epsilon_t \quad (7)$$

where, as always, $\epsilon_t \stackrel{\text{i.i.d.}}{\sim} N(0, \sigma^2)$. Note that we are writing $\beta_1(t-1)$ instead of $\beta_1 t$ for aesthetic reasons. This does not change the model in any way.

The unknown parameters in this model are $\beta_0, \beta_1, \dots, \beta_{n-1}$ as well as σ . Here are main differences between the two models (5) and (7):

1. The model (5) will be typically used with a small value of k (such as 1, 2, 3, 4). This makes it a low-dimensional model. On the other hand, the number of unknown parameters in (7) equals $n+1$ which is quite large. So (7) is an example of a high-dimensional model.
2. (5) is a nonlinear model because of the presence of the parameters $c_1, \dots, c_k$. On the other hand, there are no such nonlinear parameters in (7) which makes it a linear regression model.

To summarize, (5) is a low-dimensional nonlinear regression model, while (7) is a high-dimensional linear regression model.

2.1.1 Least Squares in (7) interpolates data

Since (7) is a linear regression model, we can estimate the coefficients in the usual way by the MLE, or equivalently, least squares by minimizing

$$\sum_{t=1}^n (y_t - \beta_0 - \beta_1(t-1) - \beta_2(t-2) - \dots - \beta_{n-1}(t-(n-1)))^2 \quad (8)$$

over all $\beta_0, \dots, \beta_{n-1}$. The smallest value achievable in the above minimization will be the RSS.

Because the number of coefficients equals the number of data points, the optimization (8) will give a perfect fit to the data leading to $RSS = 0$. This is because, for any data $y_1, \dots, y_n$, it is possible to choose $\beta_0, \dots, \beta_{n-1}$ such that

$$y_t = \beta_0 + \beta_1(t-1) + \beta_2(t-2) + \dots + \beta_{n-1}(t-(n-1)) \quad (9)$$

for every $t = 1, \dots, n$. To write $\beta_0, \dots, \beta_{n-1}$ in terms of the observed data $y_1, \dots, y_n$, first plug in $t = 1$ in (9) to get

$$\beta_0 = y_1$$

Then plug $t = 2$ in (9) to get $y_2 = \beta_0 + \beta_1 = y_1 + \beta_1$ so that

$$\beta_1 = y_2 - y_1.$$

Then plug $t = 3$ in (9) to get $y_3 = \beta_0 + 2\beta_1 + \beta_2$. Replacing $\beta_0 = y_1$ and $\beta_1 = y_2 - y_1$, we obtain

$$\beta_2 = (y_3 - y_2) - (y_2 - y_1)$$

Continuing this way plugging $t = 4, 5, \dots, n$, we get

$$\beta_t = (y_{t+1} - y_t) - (y_t - y_{t-1}).$$

Therefore the least squares parameter estimates for the model (7) are given by:

$$\beta_0 = y_1 \quad \beta_1 = y_2 - y_1 \quad \beta_j = (y_{j+1} - y_j) - (y_j - y_{j-1})$$

for $j = 2, \dots, n-1$. This will lead to fitted values which coincide exactly with the observed data. Also the MLE of σ will be zero. The unbiased estimate of σ (that we previously used in linear regression) will not exist because it will equal $\sqrt{RSS/(n-p)}$ with $p = n$.

To summarize, least squares in the model (7) will overfit the data, and will not produce any useful insight from the data.

To produce useful estimates in cases where the MLE overfits, one employs the idea of regularization.

2.2 High-dimensional Linear Model

We introduced the following model for a given time series $y_1, \dots, y_n$:

$$y_t = \beta_0 + \beta_1(t-1) + \beta_2(t-2) + \dots + \beta_{n-1}(t-(n-1)) + \epsilon_t \quad (10)$$

where, as always, $\epsilon_t \stackrel{\text{i.i.d.}}{\sim} N(0, \sigma^2)$. Here $(t-c) = (t-c)_+$ equals 0 if $t \leq c$ and equals $(t-c)$ if $t > c$.

The unknown parameters in this model are $\beta_0, \beta_1, \dots, \beta_{n-1}$ as well as σ . This is a high-dimensional model because the number of parameters (which is $n+1$) is larger than the sample size n .

2.3 Two alternative representations of (10)

There are two alternative ways of writing the model (10). The first one is

$$y_t = \mu_t + \epsilon_t \quad \text{with } \epsilon_t \stackrel{\text{i.i.d.}}{\sim} N(0, \sigma^2) \quad (11)$$

and

$$\mu_t = \beta_0 + \beta_1(t-1) + \beta_2(t-2) + \dots + \beta_{n-1}(t-(n-1)). \quad (12)$$

The β 's can be written in terms of μ_t as follows: $\beta_0 = \mu_1$, $\beta_1 = \mu_2 - \mu_1$, and

$$\beta_t = (\mu_{t+1} - \mu_t) - (\mu_t - \mu_{t-1}) \quad \text{for } t = 2, \dots, n-1.$$

In (11), μ_t can be interpreted as the underlying trend present in the data.

The second way of writing (10) is in regression form:

$$y = X\beta + \epsilon$$

where

$$X = \begin{pmatrix} 1 & 0 & 0 & \cdot & \cdot & \cdot & 0 \\ 1 & 1 & 0 & \cdot & \cdot & \cdot & 0 \\ 1 & 2 & 1 & \cdot & \cdot & \cdot & 0 \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ 1 & n-1 & n-2 & \cdot & \cdot & \cdot & 1 \end{pmatrix} \quad \text{and } \beta = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \cdot \\ \cdot \\ \cdot \\ \beta_{n-1} \end{pmatrix}.$$

2.4 (Unregularized) Least Squares

The unregularized least squares estimates of $\beta_0, \dots, \beta_{n-1}$ are obtained by minimizing the RSS:

$$\sum_{t=1}^n (y_t - \beta_0 - \beta_1(t-1) - \beta_2(t-2) - \dots - \beta_{n-1}(t-(n-1)))^2$$

over all $\beta_0, \dots, \beta_{n-1}$. This gives

$$\hat{\beta}_0 = y_1 \quad \hat{\beta}_1 = y_2 - y_1 \quad \hat{\beta}_t = (y_{t+1} - y_t) - (y_t - y_{t-1})$$

for $t = 2, \dots, n-1$. This will lead to the estimated trend function $\hat{\mu}_t = y_t$ for all t , and the smallest RSS value is zero.

These unregularized estimates will overfit the data, and will not produce a trend estimate that is simpler than the observed data.

3 Ridge and Lasso Regularization

To produce useful estimates in cases where the MLE overfits, one employs the idea of regularization. We will discuss two ways of doing this: Ridge regularization and LASSO regularization.

The Ridge estimate of β will be denoted by $\hat{\beta}_{\text{ridge}}(\lambda)$ and is given by the minimizer of:

$$\sum_{t=1}^n (y_t - \beta_0 - \beta_1(t-1) - \beta_2(t-2) - \cdots - \beta_{n-1}(t-(n-1)))^2 + \lambda (\beta_2^2 + \beta_3^2 + \cdots + \beta_{n-1}^2). \quad (13)$$

In other words, $\hat{\beta}_{\text{ridge}}(\lambda)$ minimizes a new criterion function that is obtained by adding the penalty term $\lambda(\sum_{j=2}^{n-1} \beta_j^2)$ to the least squares criterion.

Here λ denotes a tuning parameter. Different choices of λ give rise to different ridge estimators $\hat{\beta}_{\text{ridge}}(\lambda)$. When $\lambda = 0$, the penalty term is not used in (13) so that $\hat{\beta}_{\text{ridge}}(\lambda)$ coincides with the unregularized least squares estimator. If λ is set to be very large, then the penalty term dominates the objective function (13) and then the first two components of $\hat{\beta}_{\text{ridge}}(\lambda)$ coincide with linear regression while the last $n-2$ components are simply set to zero.

The LASSO estimate of β will be denoted by $\hat{\beta}_{\text{lasso}}(\lambda)$ and is given by the minimizer of:

$$\sum_{t=1}^n (y_t - \beta_0 - \beta_1(t-1) - \beta_2(t-2) - \cdots - \beta_{n-1}(t-(n-1)))^2 + \lambda (|\beta_2| + |\beta_3| + \cdots + |\beta_{n-1}|). \quad (14)$$

In other words, $\hat{\beta}_{\text{lasso}}(\lambda)$ minimizes a new criterion function that is obtained by adding the penalty term $\lambda(\sum_{j=2}^{n-1} |\beta_j|)$ to the least squares criterion. As in the case of the ridge estimator, when $\lambda = 0$, the penalty term is not used in (14) so that $\hat{\beta}_{\text{lasso}}(\lambda)$ coincides with the unregularized least squares estimator. If λ is set to be very large, then the penalty term dominates the objective function (14) and then the first two components of $\hat{\beta}_{\text{lasso}}(\lambda)$ coincide with linear regression while the last $n-2$ components are simply set to zero.

The only difference between the ridge and lasso is in the penalty term: $\sum_j \beta_j^2$ vs $\sum_j |\beta_j|$.

Note that, in usual implementations of ridge and lasso, the penalty is usually placed on all the coefficients (with the possible exception of the intercept). Here we are only placing it on $\beta_2, \dots, \beta_{n-1}$. Because of this choice, these regularized estimates revert to linear regression when the tuning parameter is large.

3.1 Alternative Representations

Based on the alternative representations of Section 2.3, we can rewrite the optimization objectives (13) and (14) as

$$\sum_{t=1}^n (y_t - \mu_t)^2 + \lambda \sum_{t=2}^{n-1} ((\mu_{t+1} - \mu_t) - (\mu_t - \mu_{t-1}))^2 \quad (15)$$

and

$$\sum_{t=1}^n (y_t - \mu_t)^2 + \lambda \sum_{t=2}^{n-1} |(\mu_{t+1} - \mu_t) - (\mu_t - \mu_{t-1})| \quad (16)$$

We denote the minimizer of (15) by $\hat{\mu}_t$ and the minimizer of (16) by $\hat{\mu}_t^*$. The relation between $\hat{\mu}_t$ and $\hat{\mu}_t^*$ is given by

$$= \hat{\beta}_0^{\text{ridge}}(\lambda) + \hat{\beta}_1^{\text{ridge}}(\lambda)(t-1) + \sum_{j=2}^{n-1} \hat{\beta}_j^{\text{ridge}}(\lambda)(t-j).$$

Similarly the relation between $\hat{\mu}_t$ and $\hat{\mu}_t^*$ is given by

$$= \hat{\beta}_0^{\text{lasso}}(\lambda) + \hat{\beta}_1^{\text{lasso}}(\lambda)(t-1) + \sum_{j=2}^{n-1} \hat{\beta}_j^{\text{lasso}}(\lambda)(t-j).$$

The estimator $\hat{\mu}_t$ is actually known by the name Hodrick-Prescott filter in the econometrics literature (see e.g., https://en.wikipedia.org/wiki/HodrickPrescott_filter), and it is closely related to the cubic spline smoother (see e.g., https://en.wikipedia.org/wiki/Smoothing_spline).

The estimator $\hat{\mu}_t^*$ is known by the name ℓ_1 trend filter (see https://stanford.edu/~boyd/papers/l1_trend_filter.html).

Both the objective functions (15) and (16) ensure good fit to the data (because of the term $\sum_{t=1}^n (y_t - \mu_t)^2$) while also ensuring that neighboring slopes $\mu_{t+1} - \mu_t$ and $\mu_t - \mu_{t-1}$ are close to each other (this is because of the terms $\lambda \sum_{t=2}^{n-1} ((\mu_{t+1} - \mu_t) - (\mu_t - \mu_{t-1}))^2$ and $\lambda \sum_{t=2}^{n-1} |(\mu_{t+1} - \mu_t) - (\mu_t - \mu_{t-1})|$). Closeness of neighboring slopes $\mu_{t+1} - \mu_t$ and $\mu_t - \mu_{t-1}$ gives a smooth appearance to $\{\mu_t\}$. These can therefore be seen as methods for trying to fit a smooth trend function μ_t to the observed time series y_t .

3.2 Ridge vs LASSO

The LASSO estimator $\hat{\mu}_t$ is usually sparse which means that most of $\hat{\beta}_2^{\text{lasso}}(\lambda), \dots, \hat{\beta}_{n-1}^{\text{lasso}}(\lambda)$ are exactly (up to numerical precision) equal to zero. This implies that $\hat{\mu}_t$ is piecewise linear. On the other hand, $\hat{\mu}_t^*$ will not be sparse in that all the terms $\hat{\beta}_2^{\text{ridge}}(\lambda), \dots, \hat{\beta}_{n-1}^{\text{ridge}}(\lambda)$ will be nonzero (even though they may be small). This gives a smooth appearance to $\hat{\mu}_t^*$.

Some insight into the tendency of the LASSO regularization to yield exact zeroes in contrast to ridge regularization can be gained from the following two simple facts.

[Simple Ridge] Suppose y is a real number and $\lambda > 0$. Then the minimizer of

$$f(\beta) = (y - \beta)^2 + \lambda\beta^2$$

is given by

$$\hat{\beta} = \frac{y}{1 + \lambda}.$$

Proof. We just need to differentiate f and set the derivative to zero:

$$f'(\beta) = 2(\beta - y) + 2\lambda\beta = 0 \implies \beta = \frac{y}{1 + \lambda}.$$

□

[Simple LASSO] Suppose y is a real number and $\lambda > 0$. Then the minimizer of

$$f(\beta) = (y - \beta)^2 + \lambda|\beta|$$

is given by

$$\hat{\beta} = \begin{cases} y - \lambda/2 & \text{if } y > \lambda/2 \\ y + \lambda/2 & \text{if } y < -\lambda/2 \\ 0 & \text{if } -\lambda/2 \leq y \leq \lambda/2. \end{cases}$$

Proof. The derivative of f is given by:

$$f'(\beta) = \begin{cases} 2(\beta - y) + \lambda & \text{if } \beta > 0 \\ 2(\beta - y) - \lambda & \text{if } \beta < 0. \end{cases}$$

At $\beta = 0$, the function $|\beta|$ is not differentiable. We now need to set the derivative to zero. Setting to zero the expression for $f'(\beta)$ for $\beta > 0$, we get

$$2(\beta - y) + \lambda = 0 \implies \beta = y - \frac{\lambda}{2}.$$

Since this expression for $f'(\beta)$ is only valid when $\beta > 0$, we need to assume that $y > \lambda/2$.

Similarly setting to zero the expression for $f'(\beta)$ when $\beta < 0$, we get

$$2(\beta - y) - \lambda = 0 \implies \beta = y + \frac{\lambda}{2}$$

which is valid when $y + \lambda/2 < 0$ or $y < -\lambda/2$.

The above calculations show that $\hat{\beta}$ equals $y - \lambda/2$ when $y > \lambda/2$, and that $\hat{\beta}$ equals $y + \lambda/2$ when $y < -\lambda/2$. In the intermediate range $-\lambda/2 \leq y \leq \lambda/2$, check that $f'(\beta) < 0$ for $\beta < 0$ and $f'(\beta) > 0$ for $\beta > 0$. This means that f is decreasing on $(-\infty, 0)$ and then increasing on $(0, \infty)$ which implies that the minimum of f has to be achieved at 0. $\square$

From these facts, it is clear that when $y \neq 0$, the ridge minimizer will never be zero, while the lasso minimizer will equal exactly zero for all y -values in the range $[-\lambda/2, \lambda/2]$. The LASSO penalty therefore has a tendency to produce exact zeros unlike the ridge penalty.

3.3 Cross-validation for selecting λ

The behavior of $\hat{\beta}$ and $\hat{\sigma}^2$ depend crucially on the choice of the tuning parameter λ . One can visually tune λ in order to obtain $\hat{\beta}$, that is simple (not too wiggly) and which fits the data well (for example, one can start with $\lambda = 1$ and either increase or decrease λ by factors of 10 until a visually appealing trend estimate is obtained). Another popular approach is to use cross-validation.

The basic idea behind cross validation is the following. First split the total set of time points $T = \{1, \dots, n\}$ into two disjoint groups T_{train} and T_{test} . Generally T_{train} will be much larger than T_{test} (e.g., T_{train} will contain about 80% of the data and T_{test} will contain about 20% of the data). For this split, fit the model to the time indices in T_{train} and obtain $\hat{\beta}_{\text{train}}^{\text{ridge}}(\lambda)$ as the minimizer of

$$\begin{aligned} & \sum_{t \in T_{\text{train}}} (y_t - \beta_0 - \beta_1(t-1) - \beta_2(t-2) - \dots - \beta_{n-1}(t-(n-1)))^2 \\ & + \lambda (\beta_2^2 + \beta_3^2 + \dots + \beta_{n-1}^2) \end{aligned} \quad (17)$$

and $\hat{\beta}_{\text{train}}^{\text{lasso}}(\lambda)$ as the minimizer of

$$\begin{aligned} & \sum_{t \in T_{\text{train}}} (y_t - \beta_0 - \beta_1(t-1) - \beta_2(t-2) - \dots - \beta_{n-1}(t-(n-1)))^2 \\ & + \lambda (|\beta_2| + |\beta_3| + \dots + |\beta_{n-1}|) \end{aligned} \quad (18)$$

Using these estimates, predict the values of y_t for $t \in T_{\text{test}}$:

$$\hat{y}_t^{\text{ridge}}(\lambda) = \hat{\beta}_{\text{train},0}^{\text{ridge}}(\lambda) + \hat{\beta}_{\text{train},1}^{\text{ridge}}(\lambda)(t-1) + \hat{\beta}_{\text{train},2}^{\text{ridge}}(\lambda)(t-2) + \dots + \hat{\beta}_{\text{train},n-1}^{\text{ridge}}(\lambda)(t-(n-1))$$

and

$$\hat{y}_t^{\text{lasso}}(\lambda) = \hat{\beta}_{\text{train},0}^{\text{lasso}}(\lambda) + \hat{\beta}_{\text{train},1}^{\text{lasso}}(\lambda)(t-1) + \hat{\beta}_{\text{train},2}^{\text{lasso}}(\lambda)(t-2) + \cdots + \hat{\beta}_{\text{train},n-1}^{\text{lasso}}(\lambda)(t-(n-1))$$

The discrepancy between the actual values of y_t and the predicted values can be calculated as:

$$\text{Test-Error}^{\text{ridge}}(\lambda) = \sum_{t \in T_{\text{test}}} \left(y_t - \hat{y}_t^{\text{ridge}}(\lambda) \right)^2 \quad \text{and} \quad \text{Test-Error}^{\text{lasso}}(\lambda) = \sum_{t \in T_{\text{test}}} \left(y_t - \hat{y}_t^{\text{lasso}}(\lambda) \right)^2$$

This test error is for a single train-test split. One can consider multiple train-test splits and add the test errors to obtain one measure of the test error for each value of λ :

$$\text{AllSplit-Test-Error}^{\text{ridge}}(\lambda) = \sum_{\text{all splits}} \text{Test-Error}^{\text{ridge}}(\lambda)$$

and

$$\text{AllSplit-Test-Error}^{\text{lasso}}(\lambda) = \sum_{\text{all splits}} \text{Test-Error}^{\text{lasso}}(\lambda)$$

This test error over all splits would be calculated for a set of candidate λ values (e.g., $\lambda = 10^a$ for $a = -8, -7, \dots, 7, 8$) and then choose the value of λ which gives the smallest test error (this would give one choice of λ for ridge, and one choice of λ for lasso).

One common choice of selecting the splits is to take the test data in **Split** i to be the i -th 20% of the times t and training data to consist of the data for all other times t i.e., the test data in **Split** 1 corresponds to the first 20% of the data, test data in Split 2 corresponds to the second 20% of the data etc. The other choice is to take the test data to be every data point with remainder 0, 1, 2, 3, 4 when divided by 5.

This method gives 5 different train-test splits, commonly known as 5-fold cross-validation.